University of Tabuk
Faculty of Computers & Information Technology
Department of Computer Engineering

MICROPROCESSORS & EMBEDDED SYSTEMS LAB CEN-412

Project (4) Smart Home Security System 10 points

1. Project description:

This project is a Smart Door Security System that integrates multiple security components to control access to a protected area. The system uses an RFID card as the first level of authentication, followed by a numeric keypad password as the second level. Access is granted only when both the RFID card and the password are correct.

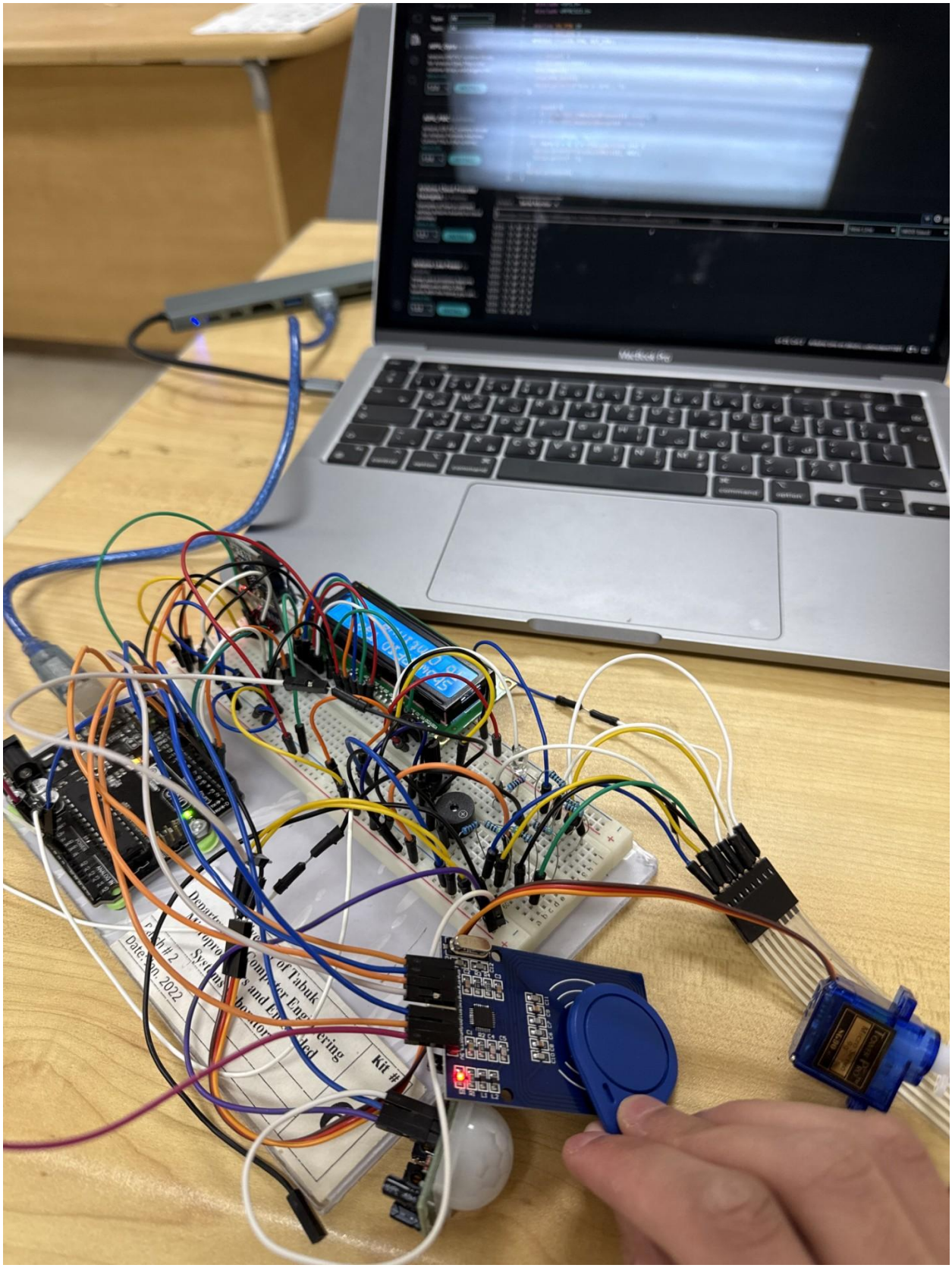
The system also uses:

- 74HC595 shift register to drive the 16x2 LCD, reducing the number of Arduino pins used.
- An analog numeric keypad for password entry.
- A servo motor to physically open and close the door.
- A buzzer for alerting when an incorrect password is entered.
- EEPROM memory for storing the password.
- LCD screen for real-time messages.

This project provides a two-factor authentication system that maximizes security while using minimal hardware resources.

2. Project schematic Diagram

In lab:



3. Algorithm and Program source code:

```
#include <SPI.h>
#include <MFRC522.h>
#include <Servo.h>
#include <EEPROM.h>
#include <LiquidCrystal_74HC595.h>

LiquidCrystal_74HC595 lcd(2, 4, 3, 1, 3, 4, 5, 6, 7);

#define KEYPAD_ANALOG_PIN A0
#define BUZZ_PIN   6
#define SERVO_PIN  8

#define RFID_RST_PIN  9
#define RFID_SS_PIN   10
MFRC522 rfid(RFID_SS_PIN, RFID_RST_PIN);

Servo doorServo;

const int PASS_LEN = 4;

const byte AUTH_TAGS[][4] = {
  {0x73, 0xBF, 0xF1, 0x0B}
};
const byte AUTH_TAG_COUNT = sizeof(AUTH_TAGS)/sizeof(AUTH_TAGS[0]);

const int KEY_VALUES[] = {
  76, 95, 135,     // 1,2,3
  80, 101,150,     // 4,5,6
  83,109,169,      // 7,8,9
  117,195,87,      // 0,#,*
  265,346,507,992  // A,B,C,D
};

const char KEYS[] = {
  '1','2','3',
  '4','5','6',
  '7','8','9',
  '0','#','*',
  'A','B','C','D'
};
```

```
#define ANALOG_TOLERANCE 8
#define NO_PRESS_VALUE 5
char getKeypadKey() {

  int v1 = analogRead(KEYPAD_ANALOG_PIN);
  if (v1 < NO_PRESS_VALUE) return '\0';

  delay(40);
  int v2 = analogRead(KEYPAD_ANALOG_PIN);

  if (abs(v1 - v2) > 8) return '\0';

  for (int i = 0; i < 16; i++) {
    if (v2 >= KEY_VALUES[i] - ANALOG_TOLERANCE &&
        v2 <= KEY_VALUES[i] + ANALOG_TOLERANCE) {

      char k = KEYS[i];

      while (analogRead(KEYPAD_ANALOG_PIN) > NO_PRESS_VALUE)
        delay(15);

      return k;
    }
  }

  return '\0';
}
void beep(unsigned long ms=120){
  digitalWrite(BUZZ_PIN,HIGH);
  delay(ms);
  digitalWrite(BUZZ_PIN,LOW);
}

void lcdCenter(const char* s, int row=0){
  int len = strlen(s);
  int pad = (16 - len)/2;
  if (pad < 0) pad=0;
  lcd.setCursor(pad, row);
  lcd.print(s);
}
```

```
void setLocked(bool locked){
  doorServo.write(locked ? 0 : 90);
}

void showMsg2(const char* l1, const char* l2, int ms=1200){
  lcd.clear();
  lcdCenter(l1,0);
  lcdCenter(l2,1);
  delay(ms);
}

bool askPassword(char* out){
  lcd.clear();
  lcdCenter("Enter Code",0);
  lcd.setCursor(0,1);
  lcd.print("> ");

  int idx=0;

  while(idx < PASS_LEN){
    char k = getKeypadKey();
    if (!k) continue;

    if (k >= '0' && k <= '9'){
      out[idx++] = k;
      lcd.print('*');
      beep(40);
    }
  }
  return true;
}

bool checkPassword(const char* c){
  const char correct[4] = {'1','2','3','4'};
  for(int i=0;i<4;i++)
    if (c[i] != correct[i]) return false;
  return true;
}

bool anyAuthorizedTagPresent(){
```

```
  if(!rfid.PICC_IsNewCardPresent()) return false;
  if(!rfid.PICC_ReadCardSerial()) return false;

  for(byte i=0;i<4;i++){
    if(rfid.uid.uidByte[i] != AUTH_TAGS[0][i])
      return false;
  }

  return true;
}

void openDoorFlow(){
  setLocked(false);
  showMsg2("Door","Opened");
  delay(5000);
  setLocked(true);
  showMsg2("Door","Closed");
}
void setup(){
  Serial.begin(115200);

  pinMode(BUZZ_PIN, OUTPUT);
  doorServo.attach(SERVO_PIN);
  setLocked(true);

  lcd.begin(16,2);
  lcdCenter("Smart Door",0);
  lcdCenter("Ready",1);

  SPI.begin();
  rfid.PCD_Init();

  delay(600);
}
void loop(){

  lcd.setCursor(0,0);
  lcd.print("Scan Your Card ");
  lcd.setCursor(0,1);
  lcd.print("               ");
```

```
  if(anyAuthorizedTagPresent()){
    beep(70);
    char pw[PASS_LEN];

    if(askPassword(pw)){
      if(checkPassword(pw)){
        showMsg2("Correct","Password");
        openDoorFlow();
      }
      else {
        showMsg2("Wrong","Password");
        beep(400);
      }
    }
  }

  delay(10);
}
```

Algorithm (Step-by-Step)

1. Initialize all modules:
    - LCD (via 74HC595)
    - RFID RC522
    - Servo motor
    - Buzzer
    - Keypad (analog input)
    - EEPROM password

2. Display: **“Scan Your Card”**

3. Wait for an RFID card:
    - If UID matches authorized card → continue
    - Else → stay waiting

4. Ask for password:
    - Read 4 digits from keypad
    - Mask each digit with “*”

5. Compare entered password with stored password:
    - If correct → display “Correct Password”
    - Unlock door (servo rotates)
    - Wait 5 seconds
    - Lock door again
    - Display “Door Closed”
    - Return to main screen

6. If password is incorrect:
    - Display “Wrong Password”
    - Activate buzzer
    - Return to main screen

4. Program description:

The program starts by including all required libraries: RFID (MFRC522), servo motor, EEPROM for storing the password, and the LiquidCrystal_74HC595 library for controlling the LCD with minimal pins.

The LCD is initialized using a shift register (74HC595) to reduce pin usage. The RFID reader is initialized using SPI communication. The servo is attached to pin 8 and set to the “locked” position.

Inside the main loop:

- The system continuously shows a message asking the user to scan their RFID card.
- When a valid card (UID: 73 BF F1 0B) is detected, the system plays a beep sound and moves to the next stage.
- The user is asked to enter a 4-digit password using an analog keypad. The code includes a robust filtering mechanism to stabilize analog readings and avoid conflicts with SPI.
- Once the password is entered, it is checked against the stored password (1234).
- If the password is correct, the servo rotates 90 degrees to open the door, waits 5 seconds, then rotates back to close it.
- If incorrect, the buzzer sounds as an alert, and the system resets.

The program is structured with several helper functions:

- **getKeypadKey()** reads the analog keypad.
- **anyAuthorizedTagPresent()** checks the RFID card.
- **askPassword()** handles user password entry.
- **checkPassword()** compares passwords.
- **openDoorFlow()** manages servo movement and LCD messages.

The system loops continuously, providing a smooth and secure two-step authentication process.